

Atividade — Árvores: revisão, quadro comparativo e analogias

Nome: Breno Silva Marinho

Disciplina: Estrutura de Dados II — Ciência da Computação — UDF

Etapa 1 — Revisão bibliográfica

Árvore geral. Organiza os dados de forma hierárquica: há uma raiz e cada nó pode ter qualquer número de filhos (zero ou mais subárvores). A propriedade fundamental é que existe um único caminho da raiz até qualquer nó, sem ciclos. Serve para hierarquias como a árvore de diretórios de um sistema de arquivos. Pode ser implementada com uma "lista de filhos" (cada nó aponta para o primeiro filho e para o próximo irmão).

Árvore binária. É uma árvore em que cada nó tem no máximo dois filhos (esquerdo e direito), e qualquer um deles pode não existir. Ela não exige ordenação dos valores. É definida recursivamente (vazia, ou raiz com duas subárvores) e é útil, por exemplo, para representar expressões aritméticas, com operandos nas folhas e operadores nos nós internos. Pode ser percorrida em pré-ordem, ordem simétrica ou pós-ordem.

Árvore binária de busca (ABB). É uma árvore binária com uma regra de ordenação: valores menores que o nó ficam na subárvore esquerda e maiores na direita (na convenção da aula, sem duplicados). Na busca, compara-se o alvo com o nó atual e desce-se por apenas um lado, descartando a outra subárvore, até achar o valor ou chegar a uma posição vazia. Na inserção, o caminho é o mesmo e o novo nó entra na posição vazia encontrada. Como não há ajuste após as alterações, a forma da árvore depende da ordem de inserção: pode ficar equilibrada (busca próxima de $O(\log n)$) ou degenerada, como uma lista (pior caso $O(n)$). Isso motiva as árvores balanceadas.

Árvore AVL. É uma ABB autobalanceada de forma estrita: em todo nó, a diferença entre as alturas das subárvores direita e esquerda (fator de balanceamento, FB) deve ficar entre -1 e +1. Após uma inserção ou remoção, recalculam-se os FBs subindo até a raiz; se algum nó chegar a ± 2 , aplica-se uma rotação. Rotações simples (LL $\rightarrow$ à direita; RR $\rightarrow$ à esquerda) resolvem quando pai e filho pendem para o mesmo lado. Rotações duplas (LR e RL) resolvem o caso de "joelho", quando os sinais do FB do pai e do filho são opostos. Por ser rígida, a busca é muito rápida, mas as alterações podem exigir várias rotações. É indicada quando há muitas consultas e poucas alterações.

Árvore rubro-negra. É uma ABB autobalanceada que usa cores (vermelho e preto) nos nós, em vez de controlar a altura exata. As propriedades são: (1) todo nó é vermelho ou preto; (2) a raiz é preta; (3) as folhas nulas são pretas; (4) um nó vermelho não tem filho vermelho; (5) todo caminho de um nó até as folhas nulas tem o mesmo número de nós pretos (altura preta). Isso garante que o maior caminho seja no máximo cerca de duas vezes o menor. Após inserir um nó vermelho, o ajuste depende da cor do tio: se o tio é vermelho, basta recolorir (e o problema pode subir); se o tio é preto, é preciso rotacionar (e recolorir). Como é menos rígida que a AVL, faz menos rotações, e por isso é boa para cargas com muitas inserções e remoções (implementações de `map` / `set` em várias linguagens).

Árvore B. É uma árvore de busca n -ária, larga e baixa, pensada para reduzir acessos a disco. Cada nó (página) guarda várias chaves ordenadas e tem um filho a mais que o número de chaves; a ordem (ou grau mínimo) define a capacidade: com ordem 5, no máximo 4 chaves e 5 filhos por nó. Todas as folhas ficam no mesmo nível. Na inserção, se o nó está cheio ocorre o split: a chave do meio sobe para o pai e o nó se divide em dois. Na remoção, se o nó fica abaixo do mínimo, pede-se chave emprestada a um irmão (redistribuição) ou fundem-se dois nós (merge). Os dados podem estar em qualquer nó. É usada em sistemas de arquivos e em índices de bancos de dados.

Árvore B+. É uma variação da B em que os dados (registros) ficam apenas nas folhas; os nós internos guardam só chaves que orientam a busca ("placas de trânsito"). As folhas são ligadas entre si numa lista encadeada, o que torna as consultas por intervalo eficientes (por exemplo, "clientes com ID de 10 a 50"): acha-se o início e segue-se pela lista. Split e merge funcionam de modo parecido com a árvore B. É a base típica de índices de bancos relacionais.

Heap. É uma estrutura baseada em árvore (geralmente uma árvore binária completa) que obrigatoriamente satisfaz a propriedade do heap, a qual determina a relação de ordem entre pais e filhos. Em um max-heap, o valor de cada pai é maior ou igual ao de seus filhos, então o maior elemento fica na raiz. Em um min-heap, o valor de cada pai é menor ou igual ao de seus filhos, então o menor elemento fica na raiz. A inserção de um elemento e a remoção da raiz custam $O(\log n)$, e a leitura da raiz custa $O(1)$. É a estrutura padrão para filas de prioridade, é a base do algoritmo de ordenação Heapsort e é muito usada para otimizar algoritmos de caminhos mínimos em grafos, como Dijkstra e Prim.

Trie. Também chamada de árvore de prefixos ou árvore digital, é uma estrutura de busca ordenada voltada à recuperação eficiente de informações, normalmente usada para armazenar e pesquisar strings. Ao contrário das ABBs clássicas, os nós não armazenam a chave: o caminho percorrido da raiz até um nó (ou folha) é que forma a chave, e cada aresta representa um caractere. Busca, inserção e remoção de uma palavra custam $O(L)$, em que L é o comprimento da palavra, o que a torna mais rápida que estruturas genéricas em buscas puramente textuais. É a principal estrutura por trás de sistemas de autocompletar em motores de busca, corretores ortográficos, roteadores de IP (busca do maior prefixo compatível) e armazenamento de dicionários complexos.

Etapa 2 – Quadro comparativo

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação	Referência
Árvore geral	Hierarquia; cada nó tem número arbitrário de filhos	Um único caminho da raiz a cada nó; sem ciclos	Inserção de subárvore como filha de um nó; percurso em pré ou pós-ordem	Árvore de diretórios	Celes e Rangel, cap. 13.2; slides da aula (14/09), slide 5
Árvore binária	Cada nó com 0, 1 ou 2 filhos (esq. e dir.)	No máximo dois filhos; sem exigência de ordem entre valores	Criação, busca em todos os nós, percursos (pré, simétrica, pós)	Árvore de expressões	Celes e Rangel, cap. 13.1; slides da aula, slide 7
ABB	Árvore binária ordenada por chave	Menores à esquerda, maiores à direita (sem duplicados)	Busca e inserção por comparações descendo um lado; sem rebalanceamento (pode degenerar)	Buscas em dados ordenados em memória; percurso em ordem gera sequência crescente	Slides da aula, slides 8 a 12; Celes e Rangel, cap. 13.1 (percurso simétrico)
AVL	ABB autobalanceada por altura	$ FB \leq 1$ em todo nó (FB = altura dir. – altura esq.)	Rotações simples (LL, RR) e duplas (LR, RL) quando FB chega a ± 2 ; propagação até a raiz	Sistemas com muitas buscas e poucas alterações	Material de estudo da disciplina (descrição de AVL); slides da aula, slide 12 (motivação)
Rubro-negra	ABB com nós coloridos	Raiz preta; vermelho sem filho vermelho; mesma altura preta em todos os caminhos	Recoloração (tio vermelho) ou rotação (tio preto) após inserção	map/s et de bibliotecas padrão; muitas inserções e remoções	Material de estudo da disciplina (descrição de rubro-negra)
B	Árvore n-ária; várias chaves ordenadas por nó; folhas no mesmo nível	Ordem m: até m-1 chaves e m filhos; mínimo de chaves por nó (exceto raiz); dados em qualquer nó	Split (chave do meio sobe) na inserção; empréstimo ou merge na remoção	Sistemas de arquivos e índices de bancos de dados	Material de estudo da disciplina (descrição de B e B+)
B+	Como a B, mas dados só nas folhas; nós internos só com chaves-guia	Folhas encadeadas em lista	Split e merge; busca por intervalo percorrendo a lista de folhas	Índices de bancos relacionais; consultas por intervalo	Material de estudo da disciplina (descrição de B e B+)
Heap	Árvore binária completa (em geral guardada em vetor)	Propriedade do heap: max-heap (pai $\geq$ filhos, maior na raiz) ou min-heap (pai $\leq$ filhos, menor na raiz)	Inserção e remoção da raiz em $O(\log n)$; leitura da raiz em $O(1)$	Filas de prioridade, Heapsort, Dijkstra e Prim	Cormen et al. (2009), cap. 6; Weiss (2011)
Trie	Árvore de prefixos; nós não guardam a chave, o caminho da raiz até o nó forma a chave; cada aresta é um caractere	Palavras com o mesmo prefixo compartilham o caminho inicial	Busca, inserção e remoção em $O(L)$, com L = tamanho da palavra	Autocompletar, corretores ortográficos, roteadores de IP (maior prefixo), dicionários	Knuth (1998), vol. 3; Weiss (2011)

Etapa 3 – Identificação por analogias

1. Estante de números reorganizada por rotações quando um lado fica alto demais.

Estrutura: árvore AVL.

Justificativa técnica: a AVL mantém $|FB| \leq 1$ e, quando um nó chega a ± 2 após inserção ou remoção,

aplica rotações (simples ou duplas) para restaurar o equilíbrio de alturas.

Limite da analogia: a estante sugere mover itens fisicamente, mas a rotação só troca ponteiros localmente e preserva a ordem da ABB. Além disso, o critério não é "um lado alto demais" de forma vaga, e sim a diferença exata de altura entre as duas subárvores de cada nó.

2. Catálogo com várias chaves por página; página cheia é dividida.

Estrutura: árvore B.

Justificativa técnica: cada nó guarda várias chaves ordenadas, com capacidade máxima definida pela ordem. Ao estourar o limite ocorre o split: a chave do meio sobe para o pai e o nó se divide em dois.

Limite da analogia: num catálogo de papel, dividir uma página não promove uma chave a um nível superior nem mantém todas as folhas na mesma profundidade. Na árvore B, o crescimento acontece pela raiz e a altura permanece uniforme.

3. Fila que mantém a tarefa de maior prioridade no topo.

Estrutura: heap (max-heap).

Justificativa técnica: a propriedade do heap garante que o pai é maior ou igual aos filhos, então o elemento de maior prioridade está sempre na raiz e é retirado primeiro.

Limite da analogia: o heap não é uma fila FIFO nem está totalmente ordenado: só se garante a relação pai-filho, e buscar um elemento qualquer custa $O(n)$. Além disso, "fila de prioridade" é o tipo abstrato de dados, e o heap é apenas uma das formas de implementá-lo.

4. Índice que percorre letras sucessivas e compartilha o início de palavras com o mesmo prefixo.

Estrutura: trie (árvore de prefixos).

Justificativa técnica: cada nível corresponde a uma posição na palavra, e palavras com prefixo comum compartilham o caminho inicial. A busca custa proporcionalmente ao tamanho da palavra, não ao número de palavras armazenadas.

Limite da analogia: um índice em papel não reflete o custo de memória da trie (muitos nós e ponteiros por caractere) nem a marcação de fim de palavra, necessária para diferenciar, por exemplo, "casa" de "casario".

5. Estrutura com cores, recolorações e rotações para controlar a altura dos caminhos de busca.

Estrutura: árvore rubro-negra.

Justificativa técnica: as propriedades das cores (vermelho sem filho vermelho e mesma altura preta em todos os caminhos) limitam o maior caminho a no máximo cerca de duas vezes o menor, e recoloração e rotação corrigem as violações após alterações.

Limite da analogia: as cores não têm significado próprio: são apenas um bit de informação usado para impor a regra. E a altura é controlada, não minimizada, pois a árvore fica menos equilibrada que uma AVL.

6. Índice que conduz às folhas com os registros, ligadas entre si para consultas por intervalo.

Estrutura: árvore B+.

Justificativa técnica: os dados ficam apenas nas folhas, os nós internos servem só de guia, e as folhas formam uma lista encadeada, de modo que uma consulta por intervalo localiza o início e segue pela lista.

Limite da analogia: um índice remissivo de livro é estático, enquanto a B+ suporta inserções e remoções dinâmicas com split e merge. E as chaves dos nós internos são guias, não os registros em si.

7. Coleção de números em que cada nó direciona valores menores para a esquerda e maiores para a direita.

Estrutura: árvore binária de busca (ABB).

Justificativa técnica: essa é exatamente a regra de ordenação da ABB, que permite descartar uma subárvore inteira a cada comparação na busca e na inserção.

Limite da analogia: uma coleção de números não garante essa organização por si só: ela depende da ordem de inserção. Sem balanceamento, a árvore pode degenerar em lista, e a busca passa de $O(\log n)$ para $O(n)$. Também é preciso definir como tratar valores duplicados (na aula, eles são excluídos).

Referências efetivamente consultadas

- CELES, W.; RANGEL, J. L. **Capítulo 13 – Árvores**. In: *Estruturas de Dados*. PUC-Rio. (Material de apoio da disciplina.)
- VALÉRIA, Kadidja. **Árvores simples e busca básica**. Slides da aula de Estrutura de Dados II, Ciência da Computação, UDF, 14 de setembro.
- Material de estudo da disciplina sobre árvore AVL, rubro-negra, B e B+ (roteiro "O que estudar").
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 3. ed. MIT Press, 2009. (Cap. 6: Heaps, Heapsort e filas de prioridade.)
- KNUTH, D. E. **The Art of Computer Programming, Volume 3: Sorting and Searching**. 2. ed. Addison-Wesley Professional, 1998. (Terminologia e análise de tries.)
- WEISS, M. A. **Data Structures and Algorithm Analysis**. 4. ed. Pearson, 2011. (Implementação e complexidade assintótica de heaps e tries.)